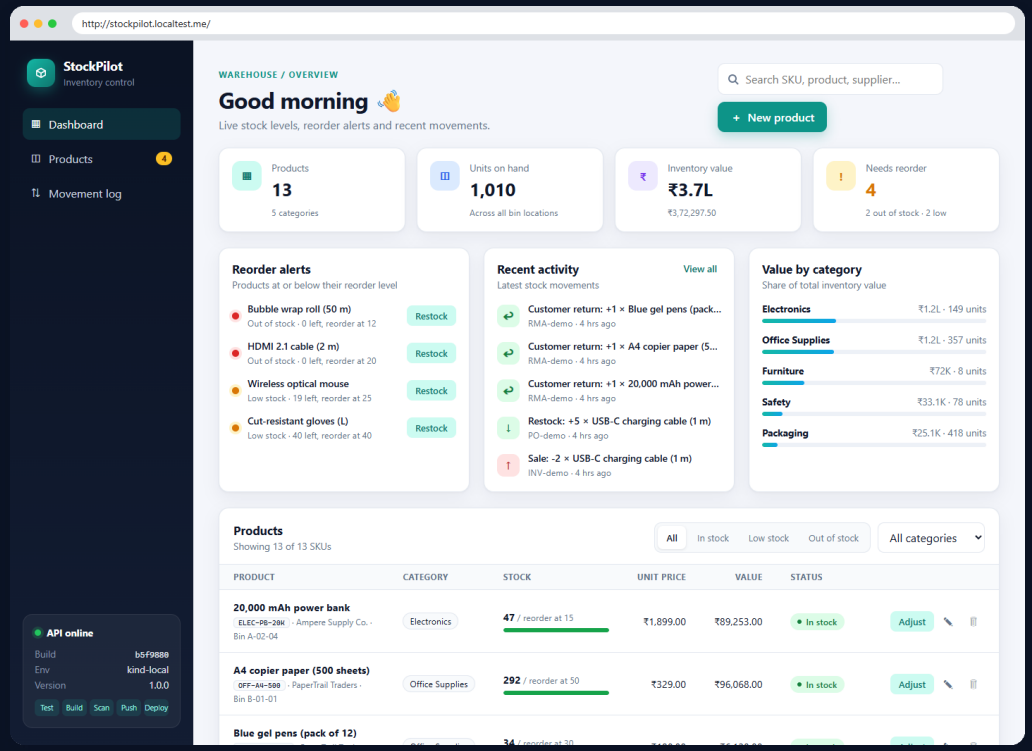
DEVOPS CAPSTONE PROJECT

StockPilot

Inventory management, from laptop to monitored Kubernetes

Varun Mundada · 24BCS10326

github.com/milesmoralis2411/devops_project_24bcs10326



FastAPI · React · PostgreSQL · Docker · GitHub Actions · Trivy · Terraform · AWS EKS · Helm · Prometheus · Grafana

THE APPLICATION

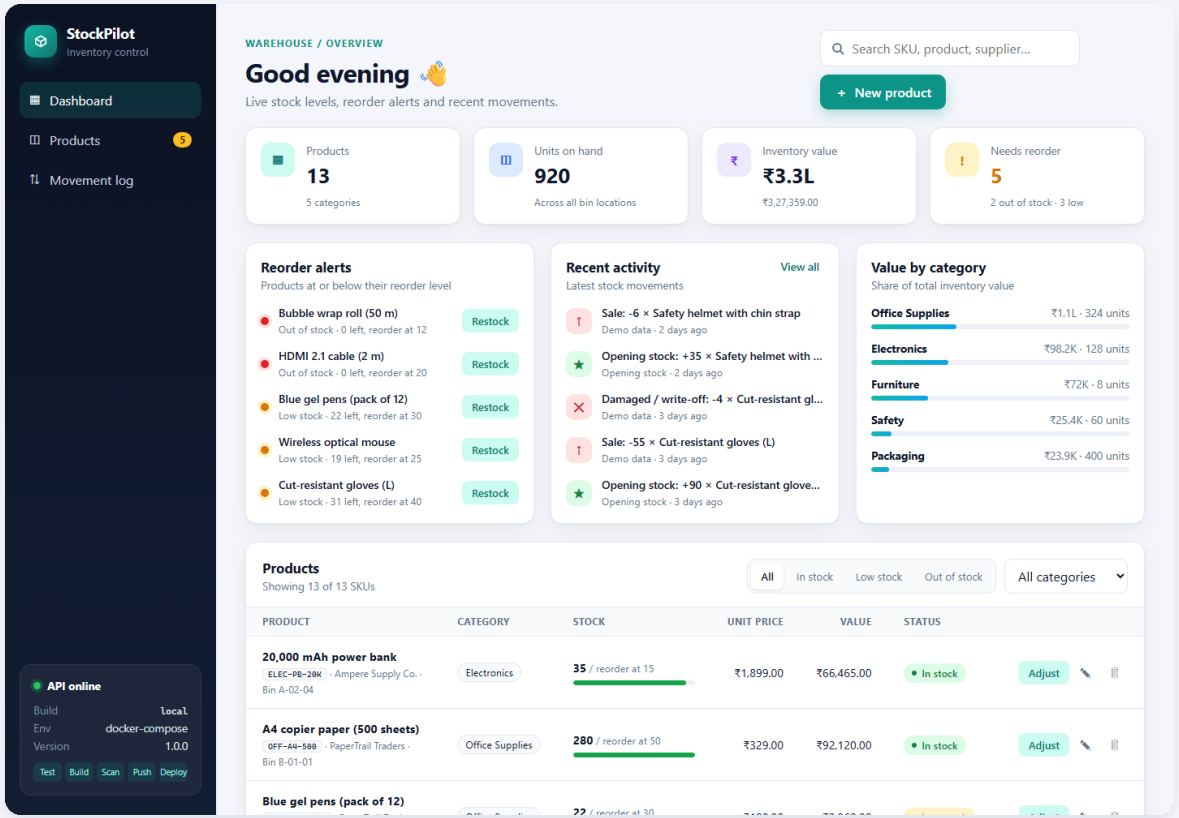
An inventory system with an audit trail

Product catalogue
SKU, supplier, bin location, unit price and reorder level for every item

Stock movements
Receipts, sales, customer returns, damage write-offs and stock counts

Reorder alerts
Items at or below their reorder level, with one-click restock

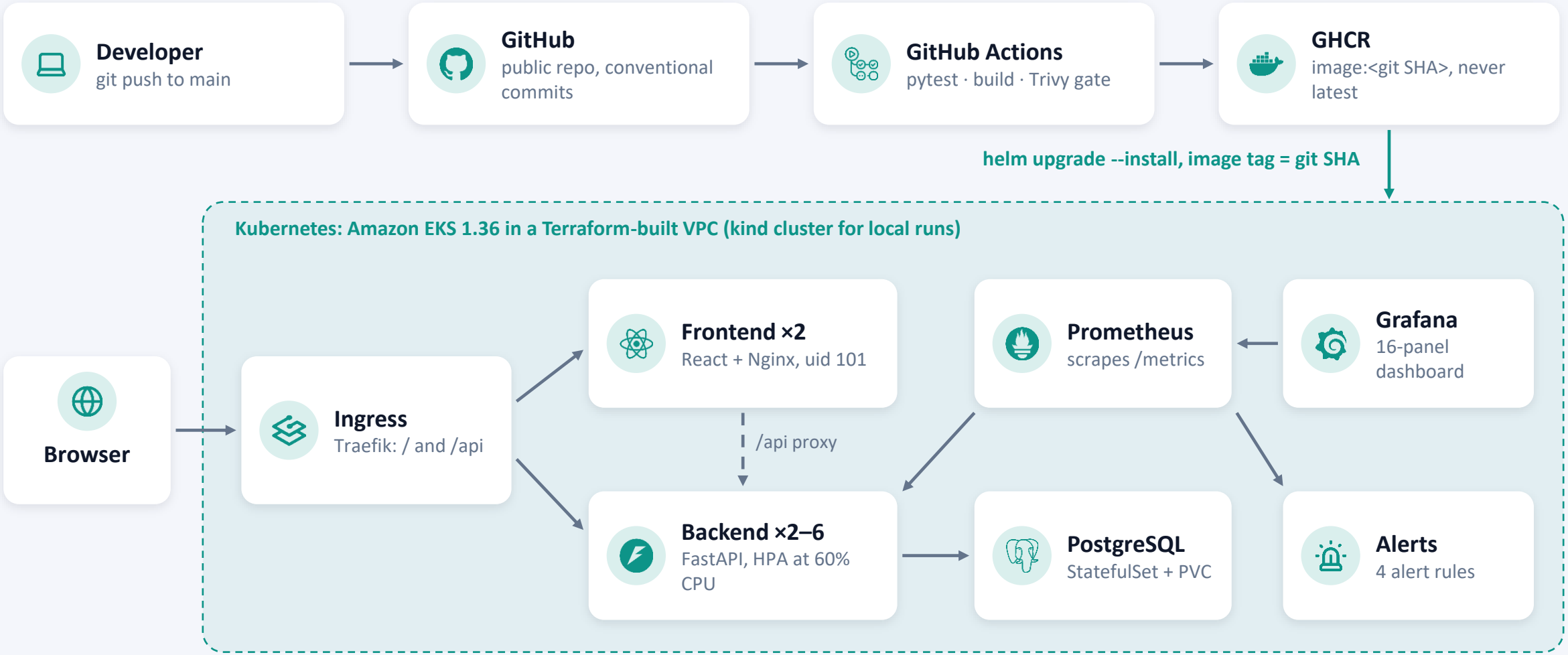
Immutable movement log
Quantity changes only through logged adjustments, so every level can be explained



Dashboard: KPIs, reorder alerts, activity feed and inventory value by category

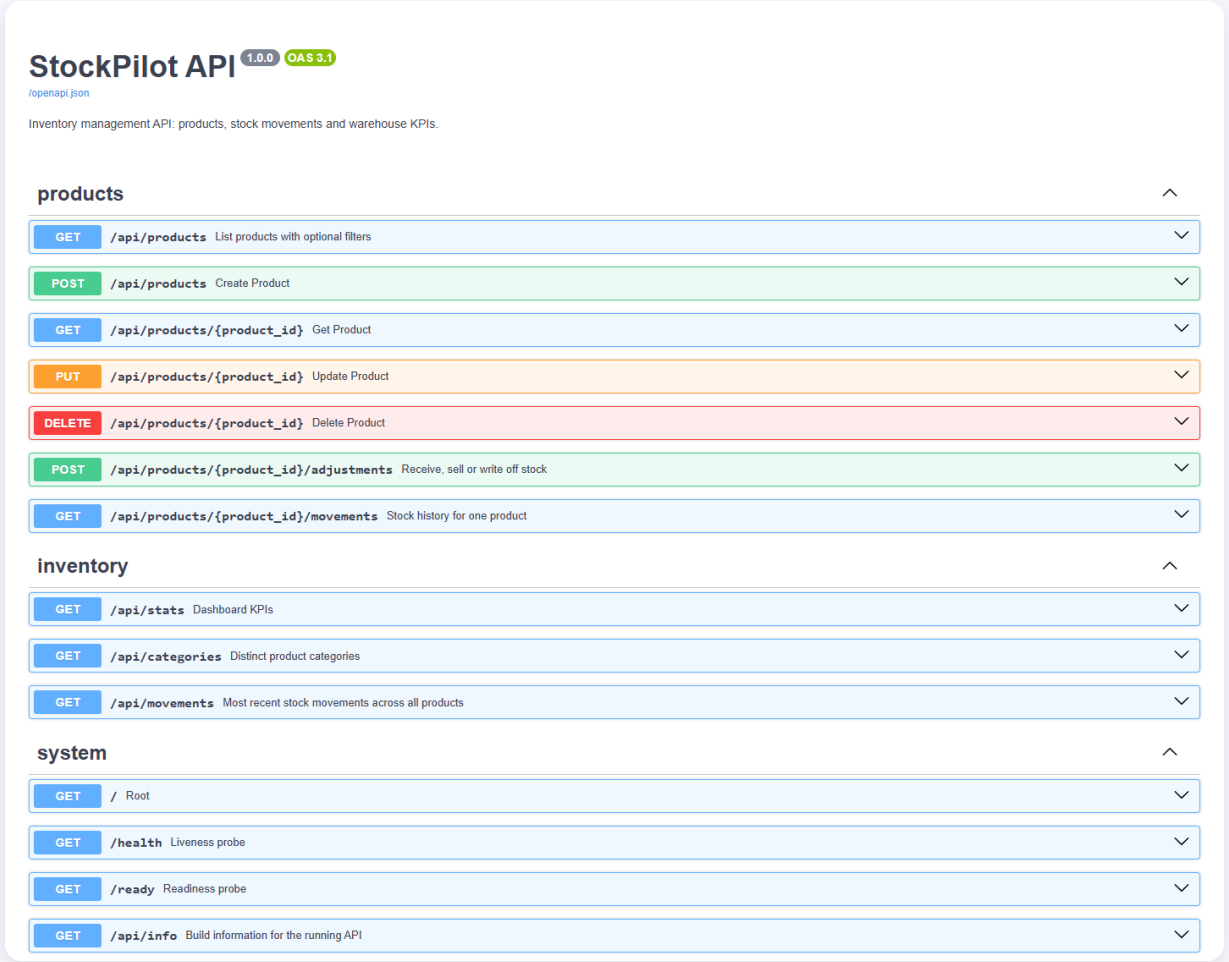
ARCHITECTURE

One repository, the whole delivery path



M1 · APPLICATION

FastAPI backend with real domain rules



11
REST endpoints under /api

2
Alembic migrations

409
returned on oversell

Rules enforced by the API

- Stock changes only through adjustments, each one logged
- Overselling returns 409, with a row lock against races
- Unique, normalised SKUs; bad input returns 422
- /health checks the process, /ready checks the database
- Migrations run once at start-up under an advisory lock
- Status is derived: IN_STOCK, LOW_STOCK, OUT_OF_STOCK

M2 · TESTING

51 tests, isolated from real data

51
pytest tests

99%
line coverage

2
databases in CI: SQLite and PostgreSQL

5
frontend unit tests

How the tests stay isolated

conftest.py sets DATABASE_URL before the app is imported and rebuilds the schema for every test. CI repeats the suite on a throw-away PostgreSQL container.

```
================================ tests coverage ================================
_____________ coverage: platform win32, python 3.12.7-final-0 _____________

Name                        Stmts   Miss  Cover
---------------------------------------------
app\__init__.py                 0      0   100%
app\config.py                  16      0   100%
app\db.py                      25      1    96%
app\main.py                    43      0   100%
app\metrics.py                 31      0   100%
app\models.py                  49      0   100%
app\routers\__init__.py         0      0   100%
app\routers\inventory.py       24      0   100%
app\routers\products.py        86      0   100%
app\schemas.py                105      0   100%
app\seed.py                    27      1    96%
---------------------------------------------
TOTAL                         406      2    99%
============================== 51 passed in 6.17s ==============================
```

Test modules

- test_products
- test_stock
- test_stats
- test_system
- test_migrations
- test_seed

Multi-stage, non-root images

stockpilot-backend

python:3.12-slim builder installs pinned dependencies into a venv; the runtime copies only the venv and code

uid 10001 | 306 MB | HEALTHCHECK

stockpilot-frontend

node:24-alpine runs npm ci and the Vite build; nginx-unprivileged serves dist/ with no Node in the image

uid 101 | 93.2 MB | port 8080

bash — devops_project

```
$ docker compose ps --format 'table {{.Service}}\t{{.Image}}\t{{.Status}}'
SERVICE    IMAGE                       STATUS
backend    stockpilot-backend:local    Up 23 seconds (healthy)
frontend   stockpilot-frontend:local   Up 11 seconds (healthy)
postgres   postgres:17-alpine          Up 28 seconds (healthy)
$ docker run --rm --entrypoint id stockpilot-backend:local
uid=10001(app) gid=10001(app) groups=10001(app)
$ docker run --rm --entrypoint id stockpilot-frontend:local
uid=101(nginx) gid=101(nginx) groups=101(nginx),101(nginx)
$ docker compose exec backend whoami
app
$ docker compose exec frontend whoami
nginx
$ docker image ls | grep -E 'stockpilot-(backend|frontend) +local'
REPOSITORY                  TAG                 SIZE
stockpilot-backend          local               306MB
stockpilot-frontend         local               93.2MB
$ docker history stockpilot-frontend:local --format '{{.CreatedBy}}' | head -6    # runtime stage: Nginx only, no Node
HEALTHCHECK {Test:[CMD-SHELL wget -qO- http:…
EXPOSE [8080/tcp]
USER 101
COPY /app/dist /usr/share/nginx/html # build…
COPY nginx/default.conf.template /etc/nginx/…
RUN /bin/sh -c apk upgrade --no-cache # buil…
```

Start order enforced by healthchecks: docker compose up --build

PostgreSQL 17
healthy when pg_isready answers

→

Backend
healthy after migrations (/health)

→

Frontend
Nginx at localhost:3000

Every push is tested, scanned and published

backend-test
ruff · pytest ×2 DBs · alembic check

frontend-build
npm ci · npm test · vite build

iac-validate
terraform test · helm lint

→

build-scan-push matrix: backend, frontend

docker build | Trivy report | Trivy gate | push :<sha>

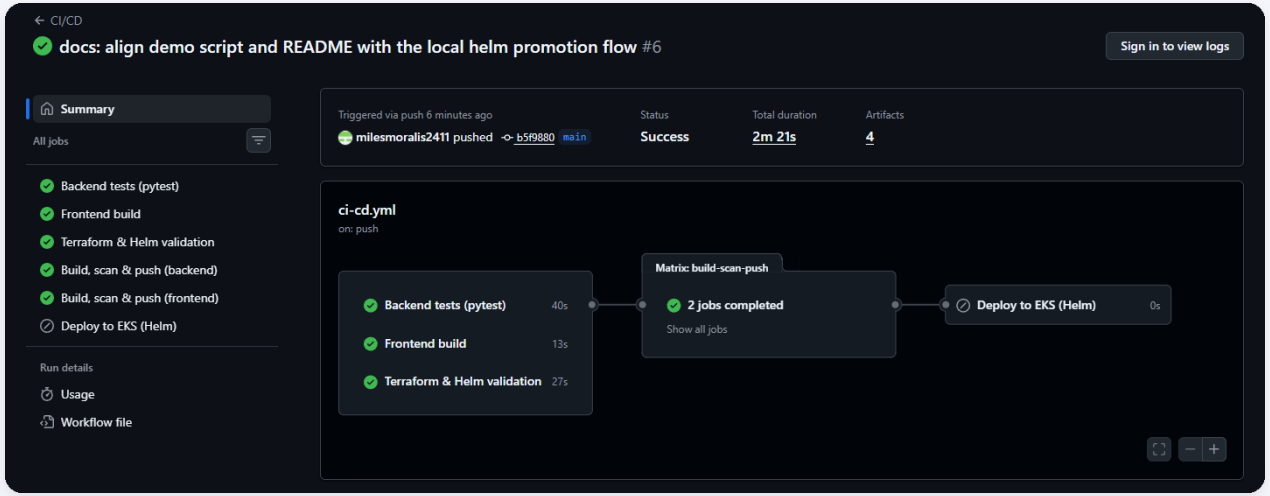
Stops on any fixable HIGH or CRITICAL CVE, before the push

→

deploy main only

helm upgrade --install with tag = SHA, --wait, helm test. AWS access via GitHub OIDC, no stored keys.

Skipped: this AWS account blocks IAM OIDC providers, so CI-built images are promoted with Helm.



- Runs on push and pull request to main, or manually
- A failing test stops the run before any image is built
- Images tagged with the full commit SHA, never latest
- Actions pinned to commit SHAs; least-privilege token
- Public repo with conventional commits (feat, fix, test, ci)

M6 · SECURITY

Trivy gate: found, explained, fixed

FIRST RUN 13 MEDIUM/LOW findings: gate passed, fixed anyway

```
Python (python-pkg)
==================
Total: 12 (UNKNOWN: 0, LOW: 2, MEDIUM: 10, HIGH: 0, CRITICAL: 0)

┌──────────────┬─────────────────┬──────────┬────────┬───────────────────┬───────────────┬──────────────────────────────────────────────────────────────┐
│   Library    │  Vulnerability  │ Severity │ Status │ Installed Version │ Fixed Version │                            Title                             │
├──────────────┼─────────────────┼──────────┼────────┼───────────────────┼───────────────┼──────────────────────────────────────────────────────────────┤
│ pip (METADATA) │ CVE-2025-8869 │ MEDIUM   │ fixed  │ 25.0.1            │ 25.3          │ pip: pip missing checks on symbolic link extraction          │
│              │                 │          │        │                   │               │ https://avd.aquasec.com/nvd/cve-2025-8869                    │
│              ├─────────────────┤          │        │                   ├───────────────┼──────────────────────────────────────────────────────────────┤
│              │ CVE-2026-13346  │          │        │                   │ 26.2.0        │ pip: pip: Arbitrary file installation via malicious package │
│              │                 │          │        │                   │               │ indexes                                                      │
│              │                 │          │        │                   │               │ https://avd.aquasec.com/nvd/cve-2026-13346                   │
│              ├─────────────────┤          │        │                   ├───────────────┼──────────────────────────────────────────────────────────────┤
│              │ CVE-2026-3219   │          │        │                   │ 26.1          │ pip: pip: Incorrect file installation due to improper        │
│              │                 │          │        │                   │               │ archive handling                                             │
│              │                 │          │        │                   │               │ https://avd.aquasec.com/nvd/cve-2026-3219                    │
│              ├─────────────────┤          │        │                   ├───────────────┼──────────────────────────────────────────────────────────────┤
│              │ CVE-2026-6357   │          │        │                   │               │ pip: pip: Arbitrary code execution or information disclosure │
│              │                 │          │        │                   │               │ via malicious wheel package...                               │
│              │                 │          │        │                   │               │ https://avd.aquasec.com/nvd/cve-2026-6357                    │
│              ├─────────────────┤          │        │                   ├───────────────┼──────────────────────────────────────────────────────────────┤
│              │ CVE-2026-8643   │          │        │                   │ 26.1.2        │ python-pip: Path traversal via malicious entry point name in │
│              │                 │          │        │                   │               │ pip wheel installation...                                    │
│              │                 │          │        │                   │               │ https://avd.aquasec.com/nvd/cve-2026-8643                    │
│              ├─────────────────┼──────────┤        │                   ├───────────────┼──────────────────────────────────────────────────────────────┤
│              │ CVE-2026-1703   │ LOW      │        │                   │ 26.0          │ pip: pip: Information disclosure via path traversal when     │
│              │                 │          │        │                   │               │ installing crafted wheel archives...                         │
│              │                 │          │        │                   │               │ https://avd.aquasec.com/nvd/cve-2026-1703                    │
└──────────────┴─────────────────┴──────────┴────────┴───────────────────┴───────────────┴──────────────────────────────────────────────────────────────┘
```

AFTER 0 fixable vulnerabilities at any severity

```
bash — devops_project (Trivy v0.75.0)

$ trivy image --severity HIGH,CRITICAL --ignore-unfixed --exit-code 1 stockpilot-frontend:local; echo "exit code: $?"

Report Summary

┌──────────────────────────────────────┬────────┬─────────────────┬─────────┐
│                Target                │  Type  │ Vulnerabilities │ Secrets │
├──────────────────────────────────────┼────────┼─────────────────┼─────────┤
│ stockpilot-frontend:local (alpine 3.24.2) │ alpine │        0        │    -    │
└──────────────────────────────────────┴────────┴─────────────────┴─────────┘

Legend:
- '-': Not scanned
- '0': Clean (no security findings detected)

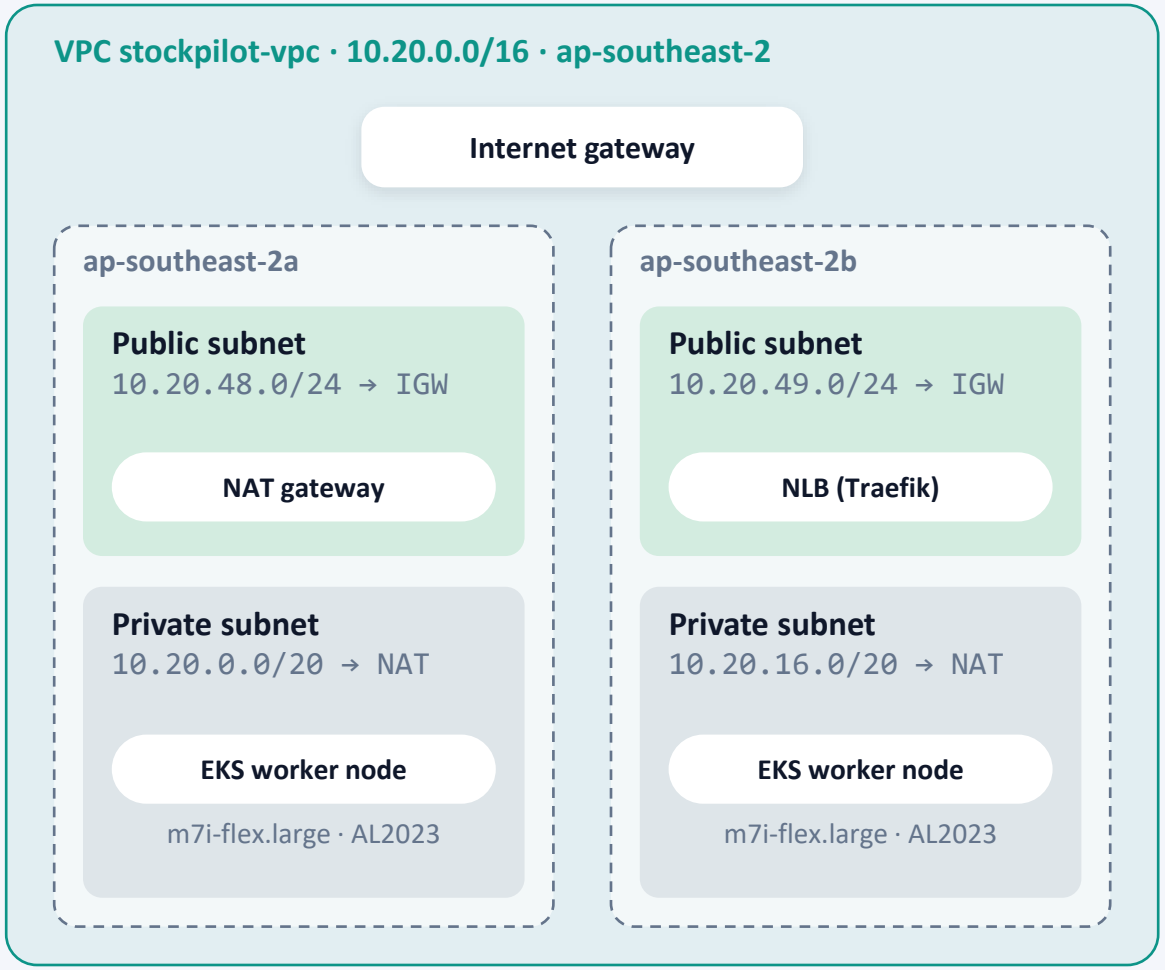
exit code: 0
$ trivy image --severity HIGH,CRITICAL --ignore-unfixed --exit-code 1 stockpilot-backend:local > /dev/null; echo "exit code: $?"
exit code: 0
$ trivy image --ignore-unfixed --exit-code 1 stockpilot-backend:local > /dev/null; echo "exit code: $?"    # ALL severities
exit code: 0
$ trivy image --ignore-unfixed --exit-code 1 stockpilot-frontend:local > /dev/null; echo "exit code: $?"   # ALL severities
exit code: 0
```

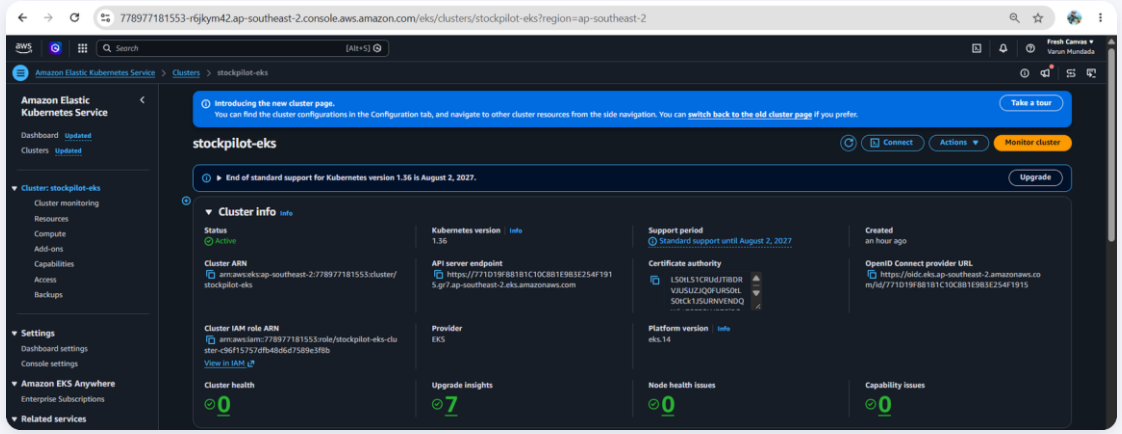
CVE-2025-8869 (pip)
pip removed from the backend runtime image

CVE-2026-85091 (zlib)
apk upgrade in the frontend runtime stage

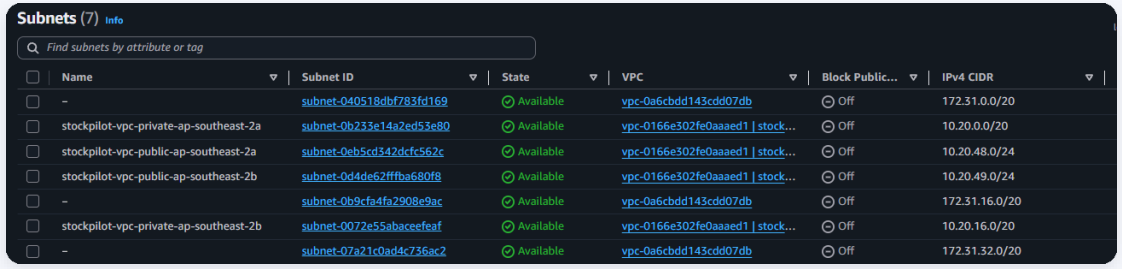
Gate on both images
HIGH,CRITICAL · fixable only · exit code 1

AWS VPC and EKS, built from code





EKS console: stockpilot-eks Active, Kubernetes 1.36



VPC console: 2 public + 2 private subnets across two AZs

Managed node group 2 × m7i-flex.large · add-ons: VPC CNI, CoreDNS, kube-proxy, Pod Identity agent, EBS CSI driver

Validate, plan, apply, destroy on real AWS

1 Validate
fmt, init, validate; terraform test with mocked AWS

→

2 Plan
64 to add, 0 to change, 0 to destroy

→

3 Apply
58 resources; EKS Active, 2 nodes Ready

→

4 Destroy
58 destroyed; the sweep finds nothing left

`terraform plan -out tfplan`

```
Plan: 64 to add, 0 to change, 0 to destroy.

Changes to Outputs:
  + aws_region            = "ap-southeast-2"
  + cluster_endpoint      = (known after apply)
  + cluster_name          = "stockpilot-eks"
  + cluster_version       = "1.36"
  + configure_kubectl     = "aws eks update-kubeconfig --region ap-southeast-2 --name stockpilot-eks"
  + github_deploy_role_arn = (known after apply)
```

`scripts/eks-teardown.sh → terraform destroy`

```
Destroy complete! Resources: 58 destroyed.
==> Leftover check - anything Kubernetes tagged for this cluster (should print nothing):
arn:aws:ec2:ap-southeast-2:778977181553:instance/i-0c520b17fbca8c1ac    arn:aws:ec2:ap-southeast-2:778977181553:instance/i-000e16d05fe23d277
# verification sweep (every aws command below was run with --region ap-southeast-2)
$ aws ec2 describe-instances --instance-ids i-0c520b17fbca8c1ac i-000e16d05fe23d277 --query 'Reservations[].Instances[].[InstanceId,State.Name]' --output text
i-0c520b17fbca8c1ac     terminated
i-000e16d05fe23d277     terminated
$ aws eks list-clusters --query 'length(clusters)'
0
```

Free-plan account limits, handled in code

Only ap-southeast-2 allowed
aws_region in terraform.tfvars

Free-tier instance types only
node group on m7i-flex.large (8 GiB)

IAM OIDC providers denied
EBS CSI via EKS Pod Identity; GitHub OIDC role off

The 64-resource plan included the optional GitHub OIDC role; the final environment had 58.

One Helm chart, a self-healing release

bash — kind cluster 'stockpilot' (3 nodes)

```
$ kubectl apply -f k8s/namespace.yaml
namespace/stockpilot unchanged
$ kubectl get pods -n stockpilot -o wide
NAME                                 READY   STATUS    RESTARTS        AGE     IP            NODE                NOMINATED NODE   READINESS GATES
stockpilot-backend-7b795f49cf-55294  1/1     Running   0               118s    10.244.1.12   stockpilot-worker   <none>           <none>
stockpilot-backend-7b795f49cf-phxhn  1/1     Running   0               103s    10.244.2.12   stockpilot-worker2  <none>           <none>
stockpilot-frontend-5598b7b6c5-6wlff 1/1     Running   0               118s    10.244.2.11   stockpilot-worker2  <none>           <none>
stockpilot-frontend-5598b7b6c5-qbjhj 1/1     Running   0               113s    10.244.1.13   stockpilot-worker   <none>           <none>
stockpilot-postgres-0                1/1     Running   1 (3h18m ago)   5h20m   10.244.1.4    stockpilot-worker   <none>           <none>
$ kubectl get deployments,statefulsets -n stockpilot
NAME                                 READY   UP-TO-DATE   AVAILABLE   AGE
deployment.apps/stockpilot-backend   2/2     2            2           5h48m
deployment.apps/stockpilot-frontend  2/2     2            2           5h48m

NAME                                 READY   AGE
statefulset.apps/stockpilot-postgres 1/1     5h48m
$ kubectl get pods -n stockpilot -o custom-columns=POD:.metadata.name,IMAGE:.spec.containers[0].image
POD                                  IMAGE
stockpilot-backend-7b795f49cf-55294  ghcr.io/milesmoralis2411/stockpilot-backend:b5f9880df9500fe6a916cc442d0b3ef947bbaa5e
stockpilot-backend-7b795f49cf-phxhn  ghcr.io/milesmoralis2411/stockpilot-backend:b5f9880df9500fe6a916cc442d0b3ef947bbaa5e
stockpilot-frontend-5598b7b6c5-6wlff ghcr.io/milesmoralis2411/stockpilot-frontend:b5f9880df9500fe6a916cc442d0b3ef947bbaa5e
stockpilot-frontend-5598b7b6c5-qbjhj ghcr.io/milesmoralis2411/stockpilot-frontend:b5f9880df9500fe6a916cc442d0b3ef947bbaa5e
stockpilot-postgres-0                postgres:17-alpine
```

Ingress (Traefik)
`stockpilot.localtest.me`

`/api` `/docs` `/openapi.json`
→ `stockpilot-backend:8000`

`/`
→ `stockpilot-frontend:80`

Deployments
backend + frontend, 2 replicas, maxUnavailable 0

Services
3 × ClusterIP: backend, frontend, postgres

HPA
backend 2 → 6 pods at 60% CPU

PodDisruptionBudgets
at least 1 pod per tier during drains

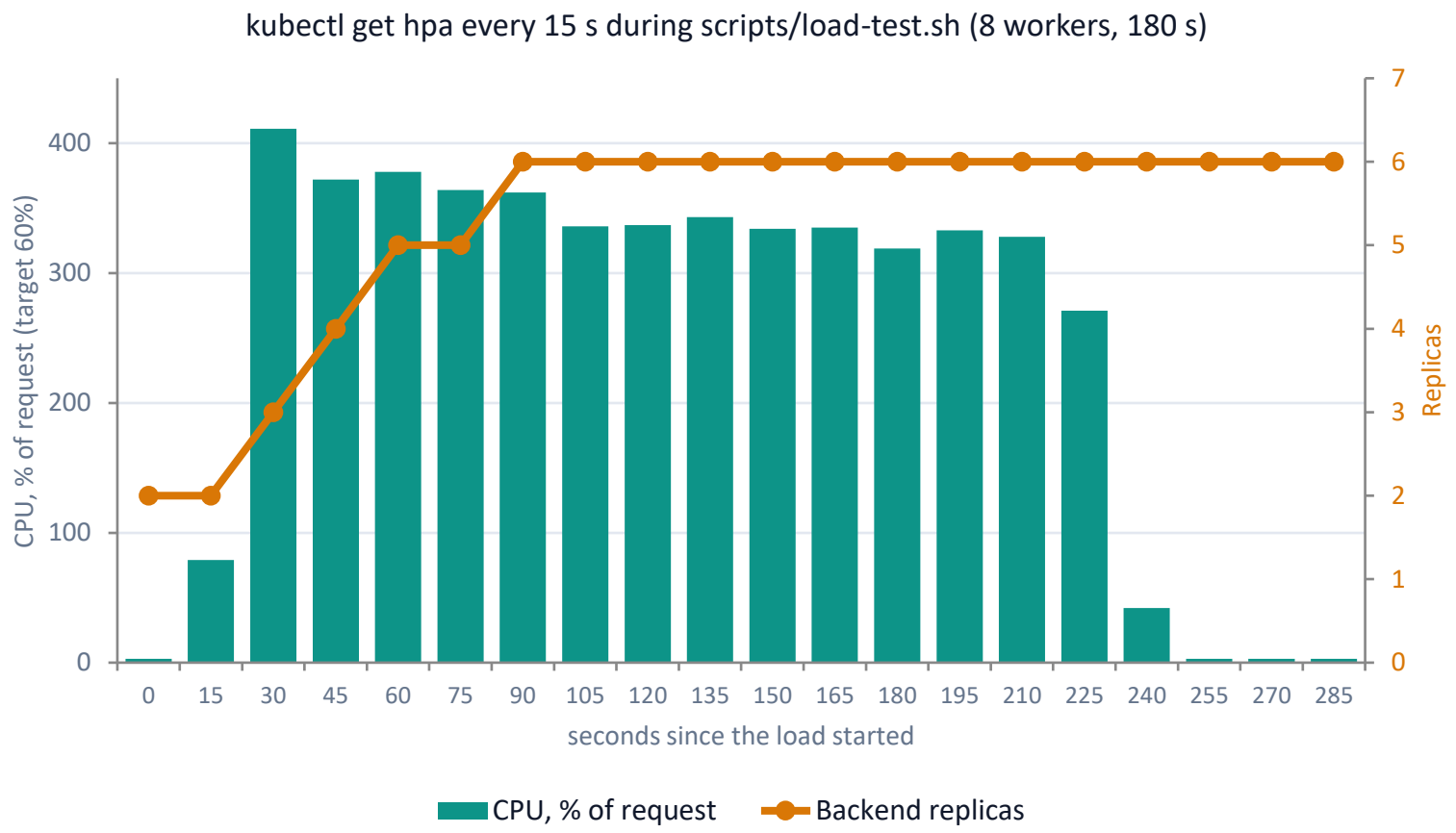
StatefulSet
PostgreSQL 17 with a PVC (gp3 on EKS)

Probes
startup, liveness /health, readiness /ready

Secret + ConfigMap
generated DB password, app settings

helm test
checks /ready, /api/stats and the UI proxy

The HPA scales 2 → 6 pods under load



2 → 6

backend pods, up to maxReplicas

411%

peak CPU against the 60% target

90 s

from the first load to 6 replicas

Metrics from the code to the dashboard

/metrics
HTTP + business metrics

→

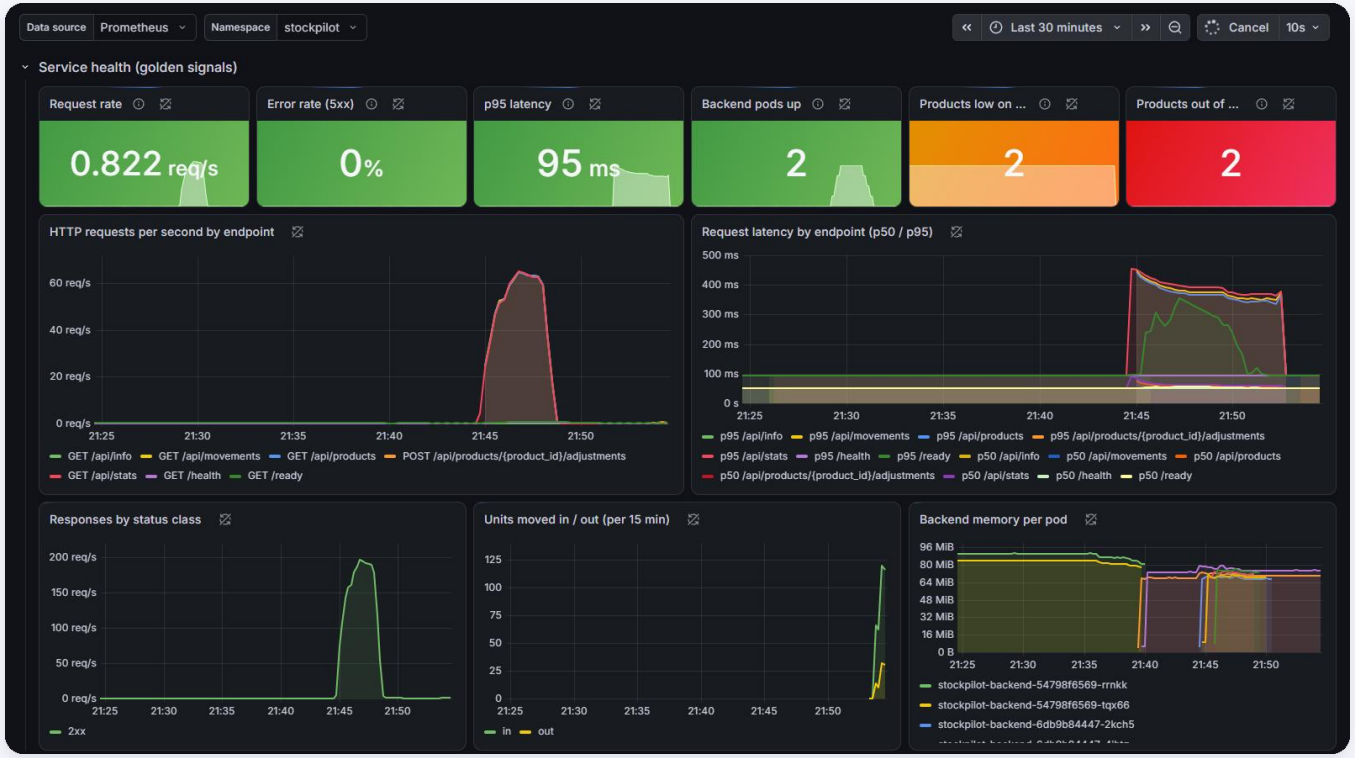
ServiceMonitor
every backend pod

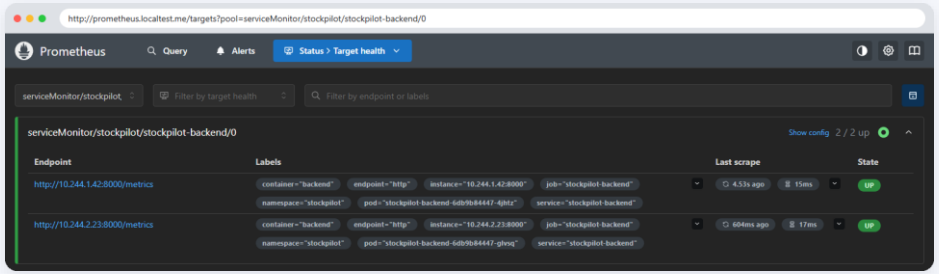
→

Prometheus
scrapes, evaluates alerts

→

Grafana
16-panel dashboard





Prometheus targets: 2 / 2 backend pods UP

Alert rules (PrometheusRule)

`StockPilotBackendDown` no backend target up

`StockPilotHighErrorRate` 5xx above 5%

`StockPilotSlowRequests` p95 above 500 ms

`StockPilotProductsOutOfStock` business alert

TROUBLESHOOTING LAB

Four broken workloads, diagnosed and fixed

`01-broken-image.yaml`

ImagePullBackOff

CAUSE

The image tag does not exist

FIX

Point at a pushed SHA tag

`02-broken-service.yaml`

No endpoints

CAUSE

The Service selector matches no pod labels

FIX

Patch the selector to the pod labels

`03-crashloop-…yaml`

CrashLoopBackOff

CAUSE

DATABASE_URL points at a host that does not exist

FIX

Load DATABASE_URL from the chart's Secret

`04-failing-readiness-…yaml`

Running, 0/1 Ready

CAUSE

The readiness probe checks the wrong port

FIX

Probe the container port

`kubectl get` → describe → logs → events → fix → verify

M10 · LIVE DEMO

Commit → pipeline → deployment update

1 Commit

Change the dashboard subtitle in frontend/src/App.jsx, commit and push to main

2 Pipeline

Tests, frontend build and IaC checks, then build → Trivy gate → push :<sha>

3 Deploy

helm upgrade with both image tags set to the new SHA; rolling update, zero downtime

4 Verify

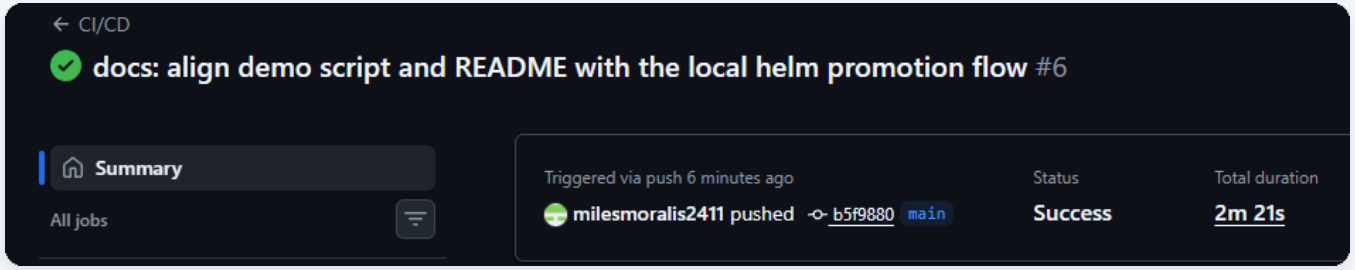
New pods Running, a new Helm revision, and the new SHA in the UI and /api/info

```
$ git commit -am "feat(ui): update dashboard subtitle" && git push
# CI: backend-test, frontend-build, iac-validate → build-scan-push → ghcr.io/…:<sha>
$ SHA=$(git rev-parse HEAD); REG=ghcr.io/milesmoralis2411
$ helm upgrade stockpilot ./helm/stockpilot -n stockpilot -f helm/stockpilot/values-dev.yaml --wait \
    --set backend.image.repository=$REG/stockpilot-backend   --set backend.image.tag=$SHA \
    --set frontend.image.repository=$REG/stockpilot-frontend --set frontend.image.tag=$SHA
$ kubectl get pods -n stockpilot && helm history stockpilot -n stockpilot
$ curl -s http://stockpilot.localtest.me/api/info       # git_sha = the new commit
```

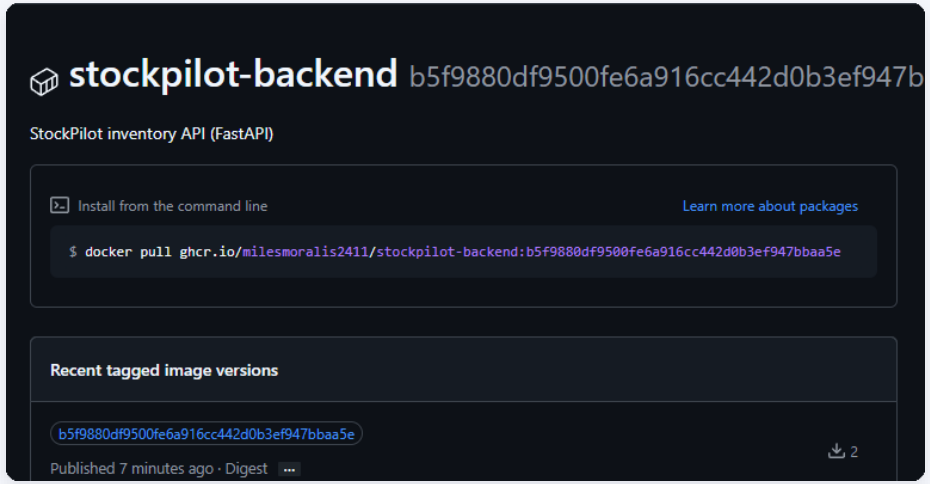
M10 · RELEASE EVIDENCE

Commit b5f9880, traced end to end

1 CI run #6 on b5f9880: every job green in 2 m 21 s



2 GHCR: image pushed with the full SHA as its tag



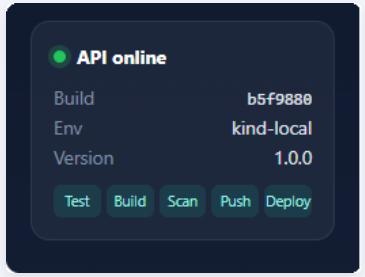
3 Every app pod runs the image tagged b5f9880…

```
$ kubectl get pods -n stockpilot -o custom-columns=POD:.metadata.name,IMAGE:.spec.containers[0].image
POD                                 IMAGE
stockpilot-backend-7b795f49cf-55294   ghcr.io/milesmoralis2411/stockpilot-backend:b5f9880df9500fe6a916cc442d0b3ef947bbaa5e
stockpilot-backend-7b795f49cf-phxhn   ghcr.io/milesmoralis2411/stockpilot-backend:b5f9880df9500fe6a916cc442d0b3ef947bbaa5e
stockpilot-frontend-5598b7b6c5-6wlff  ghcr.io/milesmoralis2411/stockpilot-frontend:b5f9880df9500fe6a916cc442d0b3ef947bbaa5e
stockpilot-frontend-5598b7b6c5-qbjhj  ghcr.io/milesmoralis2411/stockpilot-frontend:b5f9880df9500fe6a916cc442d0b3ef947bbaa5e
stockpilot-postgres-0               postgres:17-alpine
```

4 /api/info through the Ingress returns the same git_sha

```
$ curl -s http://stockpilot.localtest.me/api/info      # through the Ingress
{"service":"StockPilot API","version":"1.0.0","environment":"kind-local","git_sha":"b5f9880df9500fe6a916cc442d0b3ef947bbaa5e"}
```

5 The UI sidebar shows the running build



b5f9880

the same SHA at every stage: commit, pipeline, registry, pods, API and UI

ENGINEERING DECISIONS

Decisions behind the build

Image tag = commit SHA

Immutable, traceable releases; latest is never deployed, and rollback is helm rollback

Liveness ≠ readiness

/health skips the database, so an outage stops traffic without restart loops

Audit-first data model

Quantity changes only through logged adjustments, locked against overselling

Safe migrations

Alembic runs at start-up under an advisory lock, so N replicas migrate once

Traefik for Ingress

ingress-nginx was retired in March 2026; the Ingress stays standard networking.k8s.io/v1

Pinned everything

Lockfiles, chart versions, the Terraform lock file and SHA-pinned GitHub Actions

GRADING RUBRIC

Every module, with evidence in the repo

#	Module	Pts	Delivered	Evidence
M1	**Application**	10	FastAPI with 11 endpoints, Alembic tables, responsive React UI	`m1-*.png, Swagger`
M2	**Testing**	10	51 pytest tests, 99% coverage, isolated test database	`m2-pytest-v.png`
M3	**Git + GitHub**	5	Public repo, conventional commits, .gitignore	`m3-*.png`
M4	**Docker**	10	Multi-stage, non-root images; Compose with healthchecks	`m4-*.png`
M5	**CI/CD**	15	Test → build → scan → push to GHCR with SHA tags	`Actions run 37837734675`
M6	**Trivy**	5	HIGH/CRITICAL gate; CVEs explained and fixed	`SECURITY.md, m6-*.png`
M7	**Terraform**	15	VPC + EKS on real AWS: plan, apply, destroy	`m7-aws-*.png`
M8	**Kubernetes + Helm**	15	Chart, Ingress, HPA; 5/5 pods Running	`m8-*.png`
M9	**Observability**	10	/metrics, Prometheus UP, 16-panel Grafana	`m9-*.png`
M10	**Docs + demo**	5	README, this deck, live commit → deployment	`README.md, slides 15–16`
	Total	**100**	Index: docs/SCREENSHOTS.md · map: docs/SUBMISSION-CHECKLIST.md	

THANK YOU

Questions?

One git push tests, scans and publishes a traceable version. Helm ships it, Kubernetes keeps it healthy, Prometheus and Grafana show what it is doing.

Repository github.com/milesmoralis2411/devops_project_24bcs10326

Evidence docs/SCREENSHOTS.md · docs/SUBMISSION-CHECKLIST.md

Pipeline GitHub Actions run 37837734675

Scan for the repository